Task 11 – Role-Based Authorization (RBAC)

Goal: Control what different users can do in our application.

JWT authentication confirms who the user is. Role-based authorization determines what that user is allowed to do.

For example:

Feature	Student	Admin
View courses	Yes	Yes
Create a course	No	Yes
Update a course	No	Yes
Delete a course	No	Yes

We'll implement this without breaking your existing JWT authentication or Course CRUD API.

1 — Check the existing student data

Buddy, first we need to check how your backend currently creates student accounts so we can add roles safely.

Step 1: Open this file:

backend/services/authService.js

Step 2: Find the function that registers a student. Look for the section where the student object is created and stored.

It may look something like this:

```
const student = {
  id: someId,
  name: name,
  email: email,
  password: hashedPassword
};
```

Activity 1 — Create Role Authorization Middleware

Goal: Allow only admin users to create, update, and delete courses.

Step 1: Create a new file

Inside your backend folder, create:

backend/middleware/roleMiddleware.js

Step 2: Add this code

```
function authorizeRoles(...allowedRoles) {
  return (req, res, next) => {
    // authMiddleware must run before this middleware
    if (!req.user) {
      return res.status(401).json({
        message: "Authentication required"
      });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        message: "Access denied. Insufficient permissions."
      });
    }

    next();
  };
}
module.exports = authorizeRoles;
```

Save the file

Your backend structure should now include:

```
backend/
 middleware/
  authMiddleware.js
  roleMiddleware.js
```

Activity 2: Add a trusted role to the JWT token.

Step 1: Open authController.js

Open:

backend/controllers/authController.js

Find the login function where we previously generated the JWT.

You should see something similar to:

```
const token = jwt.sign(
 { email: student.email },
 process.env.JWT_SECRET,
 {
  subject: String(student.id),
  expiresIn: expiresIn,
 }
);
```

Step 2: Update the JWT payload

Replace only that jwt.sign() section with:

```
const token = jwt.sign(
 {
  email: student.email,
  role: "student"
 },
 process.env.JWT_SECRET,
```

```
 {
   subject: String(student.id),
   expiresIn: expiresIn
 }
);
```

What changed?

We added:

role: "student"

Save and test

Restart your backend and log in using an existing student account.

Your login should still work and return a JWT.

Don't change the course routes yet.

Activity 3: Protect Course API routes using roles.

Step 1: Open your course routes file

Open:

backend/routes/courseRoutes.js

At the top, add this import alongside your existing imports:

const authorizeRoles = require("../middleware/roleMiddleware");

Keep your existing authMiddleware import.

Step 2: Update the protected routes

Find your existing POST, PUT, and DELETE routes.

Add authorizeRoles("admin") after authMiddleware and before your existing controller function.

For example:

```
router.post(
 "/",
 authMiddleware,
 authorizeRoles("admin"),
```

```
  courseController.createCourse
);

router.put(
  "/:id",
  authMiddleware,
  authorizeRoles("admin"),
  courseController.updateCourse
);

router.delete(
  "/:id",
  authMiddleware,
  authorizeRoles("admin"),
  courseController.deleteCourse
);
```

These controller names are examples. Keep your existing controller function names and route paths.

Leave your GET routes unchanged so students can still view courses.

Step 3: Understand the result

Request	Expected result
GET courses without login	Allowed
POST course without token	401 Unauthorized
POST course with student token	403 Forbidden
POST course with valid admin token	Allowed
PUT/DELETE with student token	403 Forbidden

Step 4: Test

1. Save courseRoutes.js.
2. Restart the backend if necessary.
3. Log in as a student to get a fresh JWT.
4. Try creating a course with that token.
5. Confirm the response is 403 Forbidden.
6. Activity 4: Create a trusted admin account.

Our goal

We need an admin account that can create, update, and delete courses, while regular students can only view courses.

For security, we'll create the admin account through a backend-controlled setup process, not through public student registration.

Step 1: Open your authentication service

Open:

`backend/services/authService.js`

If that file doesn't exist, find the file containing your existing `registerStudent` and `loginStudent` functions.

Step 2: Check how students are stored

Look for one of these patterns:

```
const students = [];
```

or:

```
await Student.create(...);
```

or:

```
await pool.query(...);
```

These represent in-memory storage, a database model, or SQL storage.

Activity 4: Create an Admin Account in MongoDB

Goal: Create an admin account directly in MongoDB without allowing students to register themselves as admins.

Step 1: Check your User model

Open your existing Mongoose user model, for example:

backend/models/User.js

Make sure it has a role field:

```
role: {
  type: String,
  enum: ["student", "admin"],
  default: "student"
}
```

Add this field inside your existing schema. Don't replace your model or duplicate an existing role field.

Important: Your login system must use this same user collection. If your actual model is named Student, use that model instead.

Step 2: Add admin credentials to .env

Open backend/.env and add:

```
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=replace_with_a_strong_unique_password
```

Replace both values with your chosen admin credentials. Keep .env in .gitignore, and never share these values.

Step 3: Create the admin setup script

Create this file:

backend/scripts/createAdmin.js

```
require("dotenv").config();
```

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

// Change this import to match your existing user model.
const User = require("../models/User");

async function createAdmin() {
  try {
    const mongoUri = process.env.MONGO_URI;
    const email = process.env.ADMIN_EMAIL?.trim().toLowerCase();
    const password = process.env.ADMIN_PASSWORD;

    if (!mongoUri || !email || !password) {
      throw new Error(
        "MONGO_URI, ADMIN_EMAIL and ADMIN_PASSWORD are required"
      );
    }

    if (password.length < 12) {
      throw new Error(
        "Admin password must contain at least 12 characters"
      );
    }
```

```
  await mongoose.connect(mongoUri);

  const existingUser = await User.findOne({ email });

  if (existingUser) {
    throw new Error(
      "This email already belongs to an existing account. " +
      "Choose a different admin email."
    );
  }

  const hashedPassword = await bcrypt.hash(password, 12);

  const admin = new User({
    name: "Administrator",
    email,
    password: hashedPassword,
    role: "admin"
  });

  await admin.save();

  console.log("Admin account created successfully!");
} catch (error) {
  console.error("Admin setup failed:", error.message);
```

```
    process.exitCode = 1;
  } finally {
    await mongoose.disconnect();
  }
}

createAdmin();
```

This example assumes your model stores a name, email, password, and role, and that your database connection variable is MONGO_URI.

If your schema uses different field names, adjust them before running the script.

Also, if your Mongoose model already hashes passwords using a pre("save") hook, do not hash the password twice. Use the same password-hashing approach as your existing registration code.

Step 4: Run the script

Open a terminal in your backend folder:

node scripts/createAdmin.js

Expected output:

Admin account created successfully!

If an account with that email already exists, the script stops rather than promoting it automatically.

Step 5: Update login to use the stored role

Remember, buddy, in Activity 2 we used:

role: "student"

That was appropriate for student login, but an admin needs a JWT containing the role stored in MongoDB.

Once your login service retrieves a trusted user from the database, the JWT payload should use:

const token = jwt.sign(

```
  {
    email: user.email,
    role: user.role
  },
  process.env.JWT_SECRET,
  {
    subject: String(user._id),
    expiresIn: process.env.JWT_EXPIRES_IN || "1h"
  }
);
```

Here, user must be the authenticated Mongoose document or a trusted object returned by your login service. Never take the role from req.body.

Activity 5: Admin Login and JWT Role Verification

Goal: Allow your existing login API to generate JWT tokens with the correct role from MongoDB.

Step 1: Open authController.js

File: backend/controllers/authController.js

Find the login function where you generated the JWT in Point 10.

Step 2: Update the JWT generation

Previously, we hardcoded:

role: "student"

Now replace the JWT generation section with:

```
const token = jwt.sign(
  {
    email: student.email,
```

```
    role: student.role
  },
  process.env.JWT_SECRET,
  {
    subject: String(student._id || student.id),
    expiresIn: process.env.JWT_EXPIRES_IN || "1h"
  }
);
```

Step 3: Test admin login

Start your backend and use Postman to send a login request to your existing login endpoint.

Use the admin email and password you configured in Activity 4.

Your successful login response should contain a JWT token.

Step 4: Verify the role

For testing, you can temporarily add this line after generating the token:

```
console.log("Authenticated role:", student.role);
```

Expected output:

```
Authenticated role: admin
```

Remove the temporary logging line after testing.

Step 5: Test the protected Course API

Using the admin JWT, try creating a course.

Test	Expected result
Student token + POST course	403 Forbidden
Admin token + POST course	Course created successfully
Admin token + PUT course	Course updated successfully
Admin token + DELETE course	Course deleted successfully

The admin token must be sent using the Authorization: Bearer <token> header.

Activity 6: Show Course Management Options Only to Admins

Goal: In your React application, students should be able to view courses, while admins should see the Add, Edit, and Delete options.

Step 1: Save the role after login

Open your existing React login component.

After successful login, you should already have:

sessionStorage.setItem("token", response.data.token);

Immediately below it, add:

sessionStorage.setItem("role", response.data.student.role);

This assumes your login response includes student.role. If your response uses user.role, change it accordingly.

Step 2: Read the role in your Course component

Open the React component where you display your courses.

Inside the component, add:

```
const role = sessionStorage.getItem("role");

const isAdmin = role === "admin";
```

Step 3: Hide management buttons from students

Find your existing Add Course button and wrap it with:

```
{isAdmin && (
  <button onClick={handleAddCourse}>
    Add Course
  </button>
)}
```

Similarly, for your existing Edit and Delete buttons:

```
{isAdmin && (
  <>
```

```
    <button onClick={() => handleEdit(course)}>
      Edit
    </button>

    <button onClick={() => handleDelete(course._id)}>
      Delete
    </button>
  </>
)}
```

Important: handleAddCourse, handleEdit, and handleDelete are example names. Keep the actual functions already used in your React component.

Step 4: Clear the role on logout

In your existing logout function, add:

```
sessionStorage.removeItem("role");
```

Keep the existing code that removes the JWT token.

Step 5: Test the frontend

Login	Expected interface
Student	View courses only
Admin	View, Add, Edit, and Delete courses
Logged out	View public courses only

Security note: Hiding buttons improves the user interface, but it is not a security boundary. Your backend's authMiddleware and authorizeRoles("admin") must continue protecting POST, PUT, and DELETE requests. The role in sessionStorage is only used to control what the frontend displays.

Activity 7: Final RBAC Testing

Goal: Verify that students and admins have the correct permissions in both React and the backend.

Step 1: Start your application

Open two terminals.

Backend:

npm start

Frontend:

npm run dev

Run each command from its respective project folder. Use your existing start commands if they differ.

Step 2: Test the student account

Log in using your student credentials.

Check that you can view courses but cannot see the Add, Edit, or Delete buttons.

Step 3: Test the admin account

Log out and log in using your admin credentials.

Check that Add, Edit, and Delete buttons appear and that you can successfully perform those operations.

Step 4: Verify backend security

Use Postman to test the Course API directly.

Test	Expected result
GET courses without token	200 OK
POST course without token	401 Unauthorized
POST course with student token	403 Forbidden
POST course with admin token	Success
PUT course with student token	403 Forbidden
DELETE course with student token	403 Forbidden
PUT/DELETE with admin token	Success
Protected request with invalid token	401 Unauthorized

For the admin tests, use a test course so you don't accidentally change or delete important data.

Step 5: Final checklist

RBAC verification

Student can view courses

Student cannot see management buttons

Admin can see management buttons

Admin can create, edit, and delete courses

Student API write requests return 403

Missing or invalid tokens return 401

Task 12 — Student Course Enrolment

Goal: Allow students to enrol in courses and track which courses they have joined.

We'll continue one activity at a time, without disturbing your existing Course CRUD, login, or admin functionality.

Activity 1 — Create an Enrolment Model

Since you're using MongoDB with Mongoose, we'll create a collection that connects students with courses.

Step 1: Create this file:

backend/models/Enrollment.js

Step 2: Add the following code:

```
const mongoose = require("mongoose");

const enrollmentSchema = new mongoose.Schema(
  {
    student: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true
    },
```

```
    course: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Course",
      required: true
    },

    enrolledAt: {
      type: Date,
      default: Date.now
    },

    status: {
      type: String,
      enum: ["active", "completed"],
      default: "active"
    }
  },
  {
    timestamps: true
  }
);

// Prevent duplicate enrollments
enrollmentSchema.index(
  { student: 1, course: 1 },
```

```
  { unique: true }
);

module.exports = mongoose.model("Enrollment", enrollmentSchema);
```

Activity 2: Create the Student Enrollment API.

Our goal is to allow a logged-in student to enroll in an existing course.

Step 1 — Create the Enrollment Controller

Inside your backend, create:

backend/controllers/enrollmentController.js

Add this code:

```
const mongoose = require("mongoose");
const Enrollment = require("../models/Enrollment");
const Course = require("../models/Course");

async function enrollCourse(req, res, next) {
  try {
    // Student identity comes from verified JWT
    const studentId = req.user.sub;
    const { courseId } = req.body;

    if (!mongoose.isValidObjectId(studentId)) {
      return res.status(401).json({
        message: "Invalid student identity"
```

```
  });
}

if (!mongoose.isValidObjectId(courseId)) {
  return res.status(400).json({
    message: "Valid course ID is required"
  });
}

const course = await Course.findById(courseId);

if (!course) {
  return res.status(404).json({
    message: "Course not found"
  });
}

const existingEnrollment = await Enrollment.findOne({
  student: studentId,
  course: courseId
});

if (existingEnrollment) {
  return res.status(409).json({
    message: "Already enrolled in this course"
```

```
      });
    }

    const enrollment = await Enrollment.create({
      student: studentId,
      course: courseId
    });

    return res.status(201).json({
      message: "Course enrollment successful",
      enrollment
    });
  } catch (error) {
    if (error.code === 11000) {
      return res.status(409).json({
        message: "Already enrolled in this course"
      });
    }

    next(error);
  }
}
module.exports = { enrollCourse };
```

Step 2 — Save the file

Your backend should now contain:

backend/

models/

Enrollment.js

controllers/

enrollmentController.js

We haven't connected the controller to an API route yet, so it cannot be tested through Postman at this stage.

Your task: Create and save enrollmentController.js.

Activity 3: Create the Enrollment Route and Connect It to Express.

Goal: Allow authenticated students to enroll in courses using the API.

Step 1 — Create the Enrollment Routes File

Create:

backend/routes/enrollmentRoutes.js

Add this code:

```
const express = require("express");

const router = express.Router();


const authMiddleware = require("../middleware/authMiddleware");

const authorizeRoles = require("../middleware/roleMiddleware");


const {

  enrollCourse

} = require("../controllers/enrollmentController");
```

```
// Only logged-in students can enroll
router.post(
  "/",
  authMiddleware,
  authorizeRoles("student"),
  enrollCourse
);

module.exports = router;
```

Step 2 — Connect the Route to Express

Open your backend entry file:

backend/server.js

Add this import alongside your existing route imports:

```
const enrollmentRoutes = require("./routes/enrollmentRoutes");
```

Then add this line after your Express app is created and before your error-handling middleware:

```
app.use("/api/v1/enrollments", enrollmentRoutes);
```

Keep all your existing Course and Authentication routes unchanged.

Step 3 — Test the Enrollment API

Start your backend using your existing command.

In Postman, create a request:

POST

http://localhost:YOUR_PORT/api/v1/enrollments

Authorization

Type: Bearer Token

Token: Your student JWT from login.

Body → raw → JSON

```
{
  "courseId": "YOUR_EXISTING_COURSE_ID"
}
```

Replace YOUR_PORT and YOUR_EXISTING_COURSE_ID with actual values from your application.

Expected results

Test	Expected response
Student enrolls in an existing course	201 Created
Student enrolls in the same course again	409 Conflict
Request without JWT	401 Unauthorized
Admin attempts enrollment	403 Forbidden
Course doesn't exist	404 Not Found

Important: Your Course model must use MongoDB ObjectIds, and the student ID in the JWT must match a MongoDB user ID. If either differs, we'll need to adjust the controller.

Activity 4 — View a Student's Enrolled Courses.

Goal

When a student logs in, they should be able to see all the courses they have enrolled in.

We'll create this endpoint:

GET /api/v1/enrollments/my-courses

Step 1 — Update enrollmentController.js

Open:

backend/controllers/enrollmentController.js

Add this new function below your existing enrollCourse function.

```
async function getMyCourses(req, res, next) {
```

```
  try {
    const studentId = req.user.sub;

    if (!mongoose.isValidObjectId(studentId)) {
      return res.status(401).json({
        message: "Invalid student identity"
      });
    }

    const enrollments = await Enrollment.find({
      student: studentId
    })
      .populate("course")
      .sort({ enrolledAt: -1 });

    return res.status(200).json({
      message: "Enrolled courses fetched successfully",
      totalEnrollments: enrollments.length,
      enrollments
    });
  } catch (error) {
    next(error);
  }
}
```

Step 2 — Update the controller exports

At the bottom of the same file, replace your existing module.exports with:

```
module.exports = {
  enrollCourse,
  getMyCourses
};
```

Keep your existing enrollCourse function unchanged.

Step 3 — Update enrollmentRoutes.js

Open:

backend/routes/enrollmentRoutes.js

Update your controller import:

```
const {
  enrollCourse,
  getMyCourses
} = require("../controllers/enrollmentController");
```

Add this route above the existing enrollment POST route:

```
router.get(
  "/my-courses",
  authMiddleware,
  authorizeRoles("student"),
  getMyCourses
);
```

Keep your existing POST route unchanged.

Step 4 — Test in Postman

GET

http://localhost:YOUR_PORT/api/v1/enrollments/my-courses

Authorization: Bearer Token

Use your existing student JWT.

Expected example response:


```
{
  "message": "Enrolled courses fetched successfully",
  "totalEnrollments": 1,
  "enrollments": [
    {
      "_id": "enrollment_id",
      "student": "student_id",
      "course": {
        "_id": "course_id",
        "title": "JavaScript Basics"
      },
      "status": "active"
    }
  ]
}
```


Your actual response will contain MongoDB ObjectIds and the fields defined in your Course model.

Activity 5 — Display Enrolled Courses in React.

Goal

After a student logs in, they should be able to view all their enrolled courses on a My Courses page.

Step 1 — Create the My Courses component

Inside your React frontend, create:

src/components/MyCourses.jsx

Add this code:

```
import { useEffect, useState } from "react";
import apiClient from "../api/apiClient";

function MyCourses() {
  const [enrollments, setEnrollments] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    async function fetchMyCourses() {
      try {
        const response = await apiClient.get(
          "/enrollments/my-courses"
        );

        setEnrollments(response.data.enrollments);
      } catch (err) {
```

```
      setError("Unable to load enrolled courses");
    } finally {
      setLoading(false);
    }
  }

  fetchMyCourses();
}, []);

if (loading) {
  return <p>Loading enrolled courses...</p>;
}

if (error) {
  return <p>{error}</p>;
}

return (
  <div>
    <h2>My Enrolled Courses</h2>

    {enrollments.length === 0 ? (
      <p>You haven't enrolled in any courses yet.</p>
    ) : (
      <div>
```

```
          {enrollments.map((enrollment) => (
            <div key={enrollment._id}>
              <h3>
                {enrollment.course?.title || "Course unavailable"}
              </h3>

              <p>Status: {enrollment.status}</p>

              <p>
                Enrolled on:{" "}
                {new Date(
                  enrollment.enrolledAt
                ).toLocaleDateString()}
              </p>

              <hr />
            </div>
          ))}
        </div>
      )}
    </div>
  );
}
export default MyCourses;
```

Step 3 — Display the component

For a quick test, open src/App.jsx.

Add:

import MyCourses from "./components/MyCourses";

Then render <MyCourses /> inside your existing JSX where you want the enrolled courses to appear.

For example:

<MyCourses />

Keep your existing components and routes unchanged. Since this API is student-only, display this component only when a student is logged in.

Step 4 — Test the frontend

1. Start your React frontend using npm run dev or your existing start command.
2. Log in using a student account.
3. Open the page containing MyCourses.
4. Check whether your enrolled courses appear.

Expected result:

My Enrolled Courses

JavaScript Basics

Status:

active

Enrolled on: 26/10/2026

Illustrative example; your actual course names and dates will come from MongoDB.

Activity 6 — Add an Enroll Button to the Course List.

Goal

Students should be able to click Enroll Now on any available course. The frontend will call our existing enrollment API and display a success or error message.

Step 1 — Open your Course List component

Open the React component where you're already displaying all courses.

It might be named CourseList.jsx or Courses.jsx.

Keep your existing course display and admin management buttons unchanged.

Step 2 — Add the enrollment function

Make sure your component imports the existing Axios client:

import apiClient from "../api/apiClient";

Adjust the import path if your component is in a different folder. Don't duplicate an existing import.

Inside your component, add:

```
const handleEnroll = async (courseId) => {
  try {
    const response = await apiClient.post("/enrollments", {
      courseId
    });

    alert(response.data.message || "Enrollment successful!");
  } catch (error) {
    const message =
      error.response?.data?.message ||
      "Unable to enroll in this course";

    alert(message);
  }
};
```

Step 3 — Add the Enroll Now button

Inside the existing courses.map() section, add this button next to the course details:

```
{sessionStorage.getItem("role") === "student" && (
  <button onClick={() => handleEnroll(course._id)}>
    Enroll Now
  </button>
)}
```

This assumes your Course API returns MongoDB _id values. If it uses a different identifier, use the one expected by your enrollment controller.

Keep your existing Edit and Delete buttons for admins.

Step 4 — Test the button

Action	Expected result
Student clicks Enroll Now	Enrollment successful
Student clicks again	Already enrolled
Student opens My Courses	Newly enrolled course appears after refresh
Admin opens Course List	Enroll Now button hidden
Logged-out user opens Course List	Enroll Now button hidden

Remember: frontend button visibility is only for the user interface. Our backend still enforces student-only enrollment using JWT authentication and role authorization.

Your task

Add the handleEnroll function and the Enroll Now button to your existing Course List component.

Activity 7: Allow Students to Unenroll from Courses.

Activity 7 — Create the Unenrollment API

Goal: Allow students to remove themselves from a course they previously enrolled in.

We'll create this endpoint:

DELETE /api/v1/enrollments/:courseId

Step 1 — Update enrollmentController.js

Open:

backend/controllers/enrollmentController.js

Add this function below your existing functions:

```
async function unenrollCourse(req, res, next) {
  try {
    const studentId = req.user.sub;
    const { courseId } = req.params;

    if (
      !mongoose.isValidObjectId(studentId) ||
      !mongoose.isValidObjectId(courseId)
    ) {
      return res.status(400).json({
        message: "Invalid student or course ID"
      });
    }

    const enrollment = await Enrollment.findOneAndDelete({
      student: studentId,
      course: courseId
    });

    if (!enrollment) {
      return res.status(404).json({
        message: "You are not enrolled in this course"
      });
```

```
    }


    return res.status(200).json({
      message: "Successfully unenrolled from course"
    });
  } catch (error) {
    next(error);
  }
}
```

Step 2 — Update controller exports

At the bottom of enrollmentController.js, update:

```
module.exports = {
  enrollCourse,
  getMyCourses,
  unenrollCourse
};
```

Keep your existing functions unchanged.

Step 3 — Add the DELETE route

Open:

backend/routes/enrollmentRoutes.js

Update the controller import:

```
const {
  enrollCourse,
  getMyCourses,
  unenrollCourse
```

} = require("../controllers/enrollmentController");

Then add this route:

```
router.delete(
  "/:courseId",
  authMiddleware,
  authorizeRoles("student"),
  unenrollCourse
);
```

Step 4 — Test using Postman

DELETE

http://localhost:YOUR_PORT/api/v1/enrollments/COURSE_ID

Authorization: Bearer Token

Use your student JWT and replace COURSE_ID with an actual enrolled course's MongoDB _id.

Expected responses:

Test	Expected result
Student unenrolls from their course	200 OK
Student tries to unenroll again	404 Not Found
Request without token	401 Unauthorized
Admin attempts unenrollment	403 Forbidden

Important: This implementation permanently deletes the enrollment record. If you later need enrollment history or progress tracking, we'll change this to a status-based unenrollment approach instead.

Step 5 — Verify

After successfully unenrolling, call:

GET /api/v1/enrollments/my-courses

The removed course should no longer appear in the response.

Activity 8: Add an Unenroll Button to My Courses

Goal: Students should be able to remove a course directly from their My Enrolled Courses page without manually using Postman.

Step 1 — Open MyCourses.jsx

Open:

src/components/MyCourses.jsx

We already created this component in Activity 5.

Step 2 — Add the unenrollment function

Inside the MyCourses component, below your existing useEffect, add:

```
const handleUnenroll = async (courseId) => {
  const confirmed = window.confirm(
    "Are you sure you want to unenroll from this course?"
  );

  if (!confirmed) return;

  try {
    await apiClient.delete(`/enrollments/${courseId}`);

    setEnrollments((previous) =>
      previous.filter(
        (enrollment) => enrollment.course?._id !== courseId
      )
    );
```

```
    alert("Successfully unenrolled from the course!");
  } catch (error) {
    alert(
      error.response?.data?.message ||
      "Unable to unenroll from this course"
    );
  }
};
```

Step 3 — Add the Unenroll button

Inside your existing enrollments.map() section, find where the course name and enrollment status are displayed.

Add this button below those details:

```
<button
  onClick={() => handleUnenroll(enrollment.course._id)}
  disabled={!enrollment.course}
>
  Unenroll
</button>
```

Keep your existing course details unchanged.

Step 4 — Test the feature

1. Log in using a student account.
2. Open My Enrolled Courses.
3. Click Unenroll on a test course.
4. Confirm the action.
5. Verify that the course disappears from the list.

Activity 9: Final Student Enrollment Testing

Goal: Verify that all enrollment features work correctly before moving to Learning Progress Tracking.

Step 1 — Start your application

Run your existing backend and frontend start commands.

Step 2 — Test all enrollment features

Use this interactive checklist as you test.

Enrollment testing progress:

-Student can view available courses

Open the Course List using a student account.

-Student can enroll in a course

Click Enroll Now and confirm the enrollment succeeds.

-Duplicate enrollment is prevented

Try enrolling in the same course again; expect 409.

-My Courses displays enrolled courses

Open My Courses and confirm the new course appears.

-Student can unenroll

Click Unenroll and confirm the course disappears.

-Admin cannot enroll as a student

Test the enrollment API with an admin token; expect 403.

-Unauthenticated enrollment is blocked

Test without a JWT; expect 401.

Complete Point 12

Step 3 — Check MongoDB

Open MongoDB Compass or Atlas and inspect your enrollments collection.

Confirm that enrolling creates a document containing the student ID, course ID, enrollment date, and status. Unenrolling should remove that document under our current implementation.

Important: Our current unenrollment approach deletes records permanently. Before building progress tracking, we'll need to account for preserving learning history.

Task 13 — Learning Progress Tracking

Goal: Allow students to track how much of each enrolled course they have completed.

For example, a student who completes 6 out of 10 lessons has achieved 60% progress.

Activity 1 — Create the Learning Progress Model

Step 1: Create a new file

Inside your backend project, create:

backend/models/LearningProgress.js

Step 2: Add this code

```
const mongoose = require("mongoose");


const learningProgressSchema = new mongoose.Schema(
  {
    student: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true
    },


    course: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Course",
```

```
  required: true
},

completedLessons: {
  type: Number,
  default: 0,
  min: 0
},

totalLessons: {
  type: Number,
  required: true,
  min: 1
},

progressPercentage: {
  type: Number,
  default: 0,
  min: 0,
  max: 100
},

lastAccessedAt: {
  type: Date,
  default: Date.now
```

```
    }
  },
  {
    timestamps: true
  }
);

// One progress record per student per course
learningProgressSchema.index(
  { student: 1, course: 1 },
  { unique: true }
);

// Calculate progress automatically before saving
learningProgressSchema.pre("save", function () {
  if (this.completedLessons > this.totalLessons) {
    throw new Error(
      "Completed lessons cannot exceed total lessons"
    );
  }

  this.progressPercentage = Math.round(
    (this.completedLessons / this.totalLessons) * 100
  );
});
```

```
module.exports = mongoose.model(
  "LearningProgress",
  learningProgressSchema
);
```

Activity 2 — Create the Learning Progress API Controller.

Goal

Allow students to update their learning progress for courses they have enrolled in.

For example, if a course has 10 lessons and a student completes 4, their progress becomes 40%.

Step 1 — Create the controller file

Create:

backend/controllers/learningProgressController.js

Add this code:

```
const mongoose = require("mongoose");

const LearningProgress = require("../models/LearningProgress");
const Enrollment = require("../models/Enrollment");

// Create or update student learning progress
async function updateProgress(req, res, next) {
  try {
    const studentId = req.user.sub;
    const { courseId, completedLessons, totalLessons } = req.body;
```

```
// Validate student and course IDs
if (
  !mongoose.isValidObjectId(studentId) ||
  !mongoose.isValidObjectId(courseId)
) {
  return res.status(400).json({
    message: "Invalid student or course ID"
  });
}

// Validate lesson counts
if (
  !Number.isInteger(completedLessons) ||
  !Number.isInteger(totalLessons) ||
  totalLessons < 1 ||
  completedLessons < 0 ||
  completedLessons > totalLessons
) {
  return res.status(400).json({
    message: "Invalid lesson progress values"
  });
}

// Confirm the student is enrolled
const enrollment = await Enrollment.findOne({
```

```
  student: studentId,
  course: courseId,
  status: "active"
});

if (!enrollment) {
  return res.status(403).json({
    message: "You must be enrolled in this course"
  });
}

// Create or update progress
let progress = await LearningProgress.findOne({
  student: studentId,
  course: courseId
});

if (!progress) {
  progress = new LearningProgress({
    student: studentId,
    course: courseId,
    completedLessons,
    totalLessons
  });
} else {
```

```
      progress.completedLessons = completedLessons;
      progress.totalLessons = totalLessons;
    }

    progress.lastAccessedAt = new Date();

    await progress.save();

    return res.status(200).json({
      message: "Learning progress updated successfully",
      progress
    });
  } catch (error) {
    if (error.code === 11000) {
      return res.status(409).json({
        message: "Progress record already exists. Please retry."
      });
    }

    next(error);
  }
}

module.exports = { updateProgress };
```

Activity 3 — Create the Learning Progress API Route and Connect It to Express.

Goal

Create an API endpoint that allows authenticated students to update their learning progress.

API endpoint: PUT /api/v1/progress

Step 1 — Create the route file

Inside your backend project, create:

backend/routes/learningProgressRoutes.js

Add this code:

```
const express = require("express");

const router = express.Router();


const authMiddleware = require("../middleware/authMiddleware");

const authorizeRoles = require("../middleware/roleMiddleware");


const {

  updateProgress

} = require("../controllers/learningProgressController");


// Only authenticated students can update progress

router.put(

  "/",

  authMiddleware,

  authorizeRoles("student"),

  updateProgress
```

);

module.exports = router;

Step 2 — Connect the route to Express

Open:

backend/server.js

Add this import alongside your existing route imports:

const learningProgressRoutes = require("./routes/learningProgressRoutes");

Then add this line after creating your Express application and before your error-handling middleware:

app.use("/api/v1/progress", learningProgressRoutes);

Keep your existing authentication, course, and enrollment routes unchanged.

Step 3 — Test using Postman

PUT

http://localhost:YOUR_PORT/api/v1/progress

Authorization: Bearer Token

Use the JWT from a logged-in student.

Body → raw → JSON

```
{
  "courseId": "YOUR_ENROLLED_COURSE_ID",
  "completedLessons": 4,
  "totalLessons": 10
}
```

Replace YOUR_PORT and YOUR_ENROLLED_COURSE_ID with your actual values.

Expected response

{

```
  "message": "Learning progress updated successfully",
  "progress": {
    "completedLessons": 4,
    "totalLessons": 10,
    "progressPercentage": 40
  }
}
```

Your actual response will also contain MongoDB IDs and timestamps.

Activity 4: View Student Learning Progress

Goal: Allow logged-in students to view their learning progress for all enrolled courses.

We'll create this endpoint:

GET /api/v1/progress/my-progress

Step 1 — Update the controller

Open:

backend/controllers/learningProgressController.js

Add this function below your existing updateProgress function.

```
async function getMyProgress(req, res, next) {
  try {
    const studentId = req.user.sub;

    if (!mongoose.isValidObjectId(studentId)) {
      return res.status(401).json({
        message: "Invalid student identity"
```

```
  });
}

// Find courses in which the student is actively enrolled
const enrollments = await Enrollment.find({
  student: studentId,
  status: "active"
}).select("course");

const courseIds = enrollments.map(
  (enrollment) => enrollment.course
);

// Retrieve progress for those courses
const progressRecords = await LearningProgress.find({
  student: studentId,
  course: { $in: courseIds }
})
  .populate("course", "title")
  .sort({ updatedAt: -1 });

return res.status(200).json({
  message: "Learning progress fetched successfully",
  totalCourses: progressRecords.length,
  progress: progressRecords
```

```
    });
  } catch (error) {
    next(error);
  }
}
```

Step 2 — Update controller exports

At the bottom of the same file, replace your existing module.exports with:

```
module.exports = {
  updateProgress,
  getMyProgress
};
```

Keep your existing updateProgress function unchanged.

Step 3 — Add the GET route

Open:

backend/routes/learningProgressRoutes.js

Update your controller import:

```
const {
  updateProgress,
  getMyProgress
} = require("../controllers/learningProgressController");
```

Add this route above your existing PUT route:

```
router.get(
  "/my-progress",
  authMiddleware,
  authorizeRoles("student"),
```

```
  getMyProgress
);
```

Keep the existing PUT route unchanged.

Step 4 — Test in Postman

GET

http://localhost:YOUR_PORT/api/v1/progress/my-progress

Authorization: Bearer Token

Use your existing student JWT.

Expected response example:


```
{
  "message": "Learning progress fetched successfully",
  "totalCourses": 2,
  "progress": [
    {
      "course": {
        "_id": "course_id_1",
        "title": "JavaScript Basics"
      },
      "completedLessons": 4,
      "totalLessons": 10,
      "progressPercentage": 40
    },
    {
      "course": {
        "_id": "course_id_2",
```

```
      "title": "Python Fundamentals"
    },
    "completedLessons": 8,
    "totalLessons": 10,
    "progressPercentage": 80
  }
 ]
}
```


Step 5 — Verify

Test	Expected result
Student with progress records	200 OK with progress data
Student without progress records	200 OK with empty array
Request without JWT	401 Unauthorized
Admin attempts access	403 Forbidden

Activity 5 — Display Learning Progress in React.

Goal

Create a My Learning Progress page where students can view their enrolled courses, completed lessons, and progress percentages.

Step 1 — Create the React component

Inside your frontend, create:

src/components/MyLearningProgress.jsx

Add this code:

```
import { useEffect, useState } from "react";
import apiClient from "../api/apiClient";

function MyLearningProgress() {
```

```
const [progress, setProgress] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState("");

useEffect(() => {
  async function fetchProgress() {
    try {
      const response = await apiClient.get(
        "/progress/my-progress"
      );

      setProgress(response.data.progress || []);
    } catch (err) {
      setError("Unable to load learning progress");
    } finally {
      setLoading(false);
    }
  }

  fetchProgress();
}, []);

if (loading) {
  return <p>Loading learning progress...</p>;
}
```

```
if (error) {
  return <p>{error}</p>;
}

return (
  <div>
    <h2>My Learning Progress</h2>

    {progress.length === 0 ? (
      <p>No learning progress recorded yet.</p>
    ) : (
      progress.map((item) => (
        <div
          key={item._id}
          style={{
            border: "1px solid #ddd",
            borderRadius: "8px",
            padding: "16px",
            marginBottom: "16px"
          }}
        >
          <h3>
            {item.course?.title || "Course unavailable"}
          </h3>
```

```
            <p>
              Completed: {item.completedLessons} /{" "}
              {item.totalLessons} lessons
            </p>

            <progress
              value={item.progressPercentage}
              max="100"
              style={{ width: "100%" }}
            />

            <p>{item.progressPercentage}% Completed</p>
          </div>
        ))
      )}
    </div>
  );
}

export default MyLearningProgress;
```

Step 2 — Connect it to App.jsx

Open:

src/App.jsx

Add the import:

import MyLearningProgress from "./components/MyLearningProgress";

Then render the component in your existing student-only section:

{sessionStorage.getItem("role") === "student" && (

  <MyLearningProgress />

)}

If you already use React Router, add the component to your existing student route instead of placing it directly on the main page.

Keep your existing Course List, My Courses, and Login components unchanged.

Step 3 — Test the frontend

1. Start your backend and React frontend.
2. Log in using a student account.
3. Open the page containing My Learning Progress.
4. Verify that the course names and percentages match your MongoDB records.
5. Activity 6 — Update Learning Progress from React.

Goal

Students should be able to enter the number of lessons they have completed, click Update Progress, and see their progress bar change automatically.

Step 1 — Open `MyLearningProgress.jsx`

Open:

`src/components/MyLearningProgress.jsx`

We will update the existing component rather than create a new one.

Step 2 — Add the update function

Inside the `MyLearningProgress` component, below your existing `useEffect`, add:

const handleUpdateProgress = async (

  courseId,

```
  completedLessons,
  totalLessons
) => {
  try {
    const response = await apiClient.put("/progress", {
      courseId,
      completedLessons: Number(completedLessons),
      totalLessons: Number(totalLessons)
    });

    const updatedProgress = response.data.progress;

    setProgress((previous) =>
      previous.map((item) =>
        item.course?._id === courseId
          ? {
              ...item,
              completedLessons: updatedProgress.completedLessons,
              totalLessons: updatedProgress.totalLessons,
              progressPercentage: updatedProgress.progressPercentage
            }
          : item
      )
    );
```

```
    alert("Learning progress updated successfully!");
  } catch (error) {
    alert(
      error.response?.data?.message ||
      "Unable to update learning progress"
    );
  }
};
```

Step 3 — Create a separate progress input component

In the same file, outside the MyLearningProgress function, add:

```
function ProgressUpdater({ item, onUpdate }) {
  const [completed, setCompleted] = useState(
    String(item.completedLessons)
  );

  const handleSubmit = (event) => {
    event.preventDefault();

    const value = Number(completed);

    if (
      !Number.isInteger(value) ||
      value < 0 ||
      value > item.totalLessons
```

```
  ) {
    alert("Enter a valid number of completed lessons");
    return;
  }

  onUpdate(
    item.course._id,
    value,
    item.totalLessons
  );
};

return (
  <form onSubmit={handleSubmit}>
    <label>
      Completed Lessons:{" "}
      <input
        type="number"
        min="0"
        max={item.totalLessons}
        step="1"
        value={completed}
        onChange={(event) =>
          setCompleted(event.target.value)
        }
```

```
          required
        />
      </label>

      <button type="submit">
        Update Progress
      </button>
    </form>
  );
}
```

Step 4 — Display the update form

Inside your existing progress.map() section, below the progress bar, add:

```
{item.course && (
  <ProgressUpdater
    item={item}
    onUpdate={handleUpdateProgress}
  />
)}
```

Keep your existing course name, completed lessons, and progress bar.

Activity 7: Automatically Mark Courses as Completed

Goal: When a student reaches 100% learning progress, the enrollment status should automatically change from active to completed.

Step 1 — Open learningProgressController.js

File:

backend/controllers/learningProgressController.js

Find the existing updateProgress function.

Step 2 — Add automatic enrollment status updates

Find this existing line:

await progress.save();

Immediately below that line, add:

```
if (progress.progressPercentage === 100) {
  await Enrollment.findOneAndUpdate(
    {
      student: studentId,
      course: courseId
    },
    {
      $set: { status: "completed" }
    }
  );
}
```

Step 3 — Important adjustment

Buddy, our earlier getMyProgress API retrieves only active enrollments. If we leave it unchanged, a course could disappear from the progress page as soon as it reaches 100%.

Open the getMyProgress function and find:

```
const enrollments = await Enrollment.find({
  student: studentId,
  status: "active"
}).select("course");
```

Replace it with:

const enrollments = await Enrollment.find({

student: studentId,

status: { $in: ["active", "completed"] }

}).select("course");

This keeps completed courses visible on the learning progress page.

Step 4 — Test using Postman or React

Update an enrolled course to 100%:

{

"courseId": "YOUR_COURSE_ID",

"completedLessons": 10,

"totalLessons": 10

}

Then check MongoDB's enrollments collection.

Expected result:

{

"status": "completed"

}

Finally, refresh your React learning progress page. The course should still appear with 100% progress.

Activity 8: Display Completed Course Status in React

Goal: Show a Completed badge when a student finishes all lessons, and In Progress for courses that are not yet finished.

We'll update your existing MyLearningProgress.jsx component.

Step 1 — Open MyLearningProgress.jsx

File:

src/components/MyLearningProgress.jsx

Find the existing progress.map() section where you display the course name and progress bar.

Step 2 — Add the course status

Below the progress percentage, add:

```
<p>
  Status:{" "}
  {item.progressPercentage === 100
    ? "Completed"
    : "In Progress"}
</p>
```

This uses the progress percentage to display the status.

Step 3 — Disable updates for completed courses

Since our backend currently prevents updating a completed enrollment, we should hide the progress update form after 100% completion.

Find your existing code:

```
{item.course && (
  <ProgressUpdater
    item={item}
    onUpdate={handleUpdateProgress}
  />
)}
```

Replace it with:

```
{item.course && item.progressPercentage < 100 && (
  <ProgressUpdater
    item={item}
    onUpdate={handleUpdateProgress}
```

```
  />

)}
```

Now students cannot submit further progress updates from the UI once a course is completed.

Step 4 — Test the changes

1. Log in using a student account.
2. Open My Learning Progress.
3. Check that courses below 100% display In Progress.
4. Update a test course to 100%.
5. Verify that it displays Completed and the update form disappears.

Important: The frontend status is derived from the progress percentage. The backend remains responsible for enforcing the actual enrollment status.

Activity 9 — Create a Student Learning Progress Summary.

Goal

Display three summary cards on the student's learning progress page:

Example values only. Your application will calculate these from MongoDB data.

Step 1 — Open MyLearningProgress.jsx

File:

src/components/MyLearningProgress.jsx

We'll reuse the progress state and API that you already created. No new backend endpoint is needed for this activity.

Step 2 — Calculate the summary

Inside the MyLearningProgress component, after your existing state declarations, add:

```
const totalCourses = progress.length;


const completedCourses = progress.filter(

  (item) => item.progressPercentage === 100

).length;
```

```
const averageProgress =
  totalCourses > 0
    ? Math.round(
        progress.reduce(
          (sum, item) => sum + item.progressPercentage,
          0
        ) / totalCourses
      )
    : 0;
```

Step 3 — Display the summary cards

Inside your component's return, find:

```
<h2>My Learning Progress</h2>
```

Immediately below it, add:

```
<div
  style={{
    display: "flex",
    gap: "16px",
    flexWrap: "wrap",
    marginBottom: "24px"
  }}
>
  <div className="summary-card">
    <h3>Total Courses</h3>
```

```
    <h2>{totalCourses}</h2>
  </div>

  <div className="summary-card">
    <h3>Completed Courses</h3>
    <h2>{completedCourses}</h2>
  </div>

  <div className="summary-card">
    <h3>Average Progress</h3>
    <h2>{averageProgress}%</h2>
  </div>
</div>
```

Step 4 — Add styling

Open your existing CSS file, such as src/App.css, and add:

```
.summary-card {
  flex: 1;
  min-width: 160px;
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 10px;
  text-align: center;
  background-color: #f8fafc;
}
```

```
.summary-card h3 {
  font-size: 15px;
  color: #475569;
}

.summary-card h2 {
  font-size: 28px;
  color: #2563eb;
}
```

Make sure that CSS file is already imported into your React application.

Step 5 — Test

1. Start your frontend and backend.
2. Log in using a student account.
3. Open My Learning Progress.
4. Verify that the summary cards display.
5. Update a course's progress and check whether the average changes.
6. Complete a course and confirm the completed-course count increases.

Important: This version counts courses that have progress records. Enrolled courses without progress records aren't included yet. We'll improve that when we integrate enrollment and progress summaries.

Activity 10: Final Learning Progress Testing

Goal: Verify that the backend, MongoDB, and React frontend work together correctly.

Step 1 — Start your application

Start your backend and React frontend using your existing commands.

Log in using a student account.

Step 2 — Complete the testing checklist

Final testing progress:

-Learning progress page loads

Course names and progress bars appear correctly.

-Progress updates successfully

Change completed lessons from 4/10 to 7/10. Expect 70%.

-Invalid progress is rejected

Try 11 completed lessons out of 10. Expect 400.

-Unenrolled courses are protected

Updating a course without an active enrollment should return 403.

-100% completion works

Complete 10/10 lessons and check MongoDB enrollment status is completed.

-Completed courses remain visible

Refresh the progress page. The completed course should still appear.

-Summary cards update correctly

Verify the completed-course count and average percentage.

-JWT and role restrictions work

Missing token returns 401; admin token returns 403.

Step 3 — Check MongoDB

Open MongoDB Compass or Atlas and inspect:

- learningprogresses — check the completed lesson counts and percentages.
- enrollments — check that courses reaching 100% have status: "completed".

The exact collection name may differ if you've configured it explicitly.